PART 1: NUMBER SYSTEMS

BINARY CODES

SAAD BAIG – DIGITAL LOGIC DESIGN

The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from https://www.hasanbaig.com/resources

HOMEWORK 1 RELEASED

- Due by **September 8, 2021, 11:59 PM.**
- This assignment should be done individually.
- Scan your answer sheet and upload it on LMS before the due date.

QUIZ 1 ANNOUNCEMENT

- Quiz 1 will be conducted Saturday 10th September.
- All of Number Systems (Morris Mano 1.2 to 1.7) which will be concluded today:
  - General Number Systems
  - Number System Conversions
  - Number Complements
  - Signed Binary Numbers
  - Binary Codes (today)

BINARY CODES

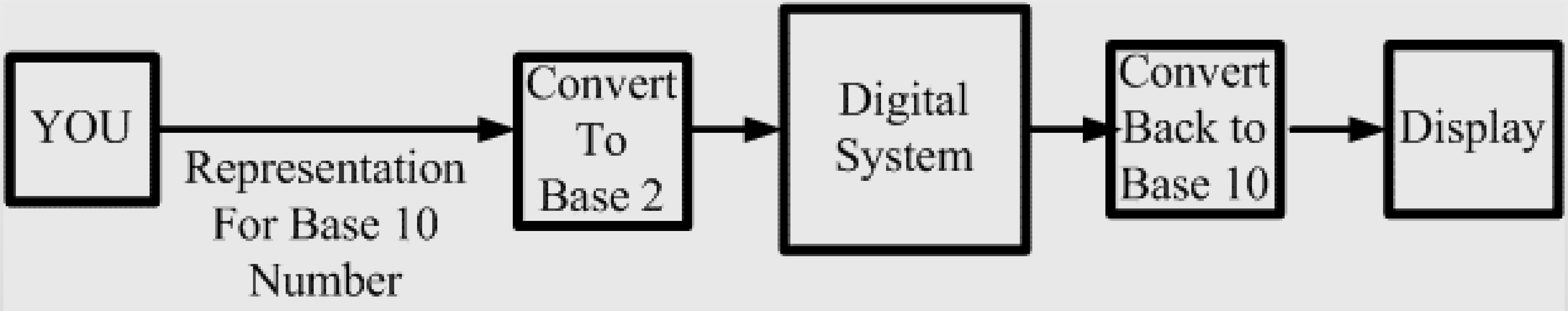
- Digital systems represent and manipulate not only binary numbers, but also many other discrete elements of information. Most of the time binary bit patterns represent encoded information rather than a number.

Any discrete element of information that is distinct among a group of quantities can be represented with a binary code (a pattern of 0's and 1's).

- *n*-bit binary code is a group of *n* bits that assumes up 2^n distinct combinations of 1's and 0's, from 0 to $2^n - 1$. Each combination represents one element of the set that is being coded.
  - Example: Set of four (2^2) elements can be coded with two bits from 0 to 3: 00, 01, 10, 11.

BINARY CODES

- We are born into a base 10 environment, but computers communicate in base 2.
- So we program computers to do the conversions for us.



BINARY CODED DECIMAL

- Express each decimal number (0 to 9) with a binary code.
- There are only ten code groups in BCD.
- How many bits will you need to represent a decimal number?
- You need 4 bits to represent the largest decimal digit 9.
- 4 bits have 16 possible combinations but only 10 are utilized, the rest of the 6 go unused.
- A number with *k* decimal digits will require 4*k* bits in BCD.

Decimal Symbol	**BCD Digit**
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BINARY CODED DECIMAL

- Although BCD use bits in their representation, they are still decimal numbers and NOT binary numbers.
  - Decimal 0 to 9 uses 4 bits. From 0 to 9, decimal and binary representation is the same.
  - Decimal 10 to 19 uses 8 bits, so the representation is NOT the same in binary.
  - Example: Decimal 15 in BCD is 0001 0101. Decimal 15 in binary is 1111.
- So the representation of a BCD number needs more bits than its equivalent binary value.
- Still, there is an advantage in the use of decimal numbers, because computer input and output data are generated by people who use the decimal system.

ADDITION IN BINARY CODED DECIMAL

1. Add the two BCD numbers, using the rules for binary addition.
2. If a 4-bit sum is equal to or less than 9, it is a valid BCD number.
3. If a 4-bit sum is greater than 9, or if a carry out of the 4-bit group is generated, it is an invalid result.
   - Add 6 (0110) to the 4-bit sum in order to skip the six invalid states.
   - If a carry results when 6 is added, simply add the carry to the next 4-bit group.

ADDITION IN BINARY CODED DECIMAL

1. 00100011 + 00010101

$$
\begin{array}{rr@{\quad}r}
 & 0010 \quad 0011 & 23 \\
+ & 0001 \quad 0101 & +\ 15 \\
\hline
 & \mathbf{0011} \quad \mathbf{1000} & 38
\end{array}
$$

2. 184 + 576 = 760

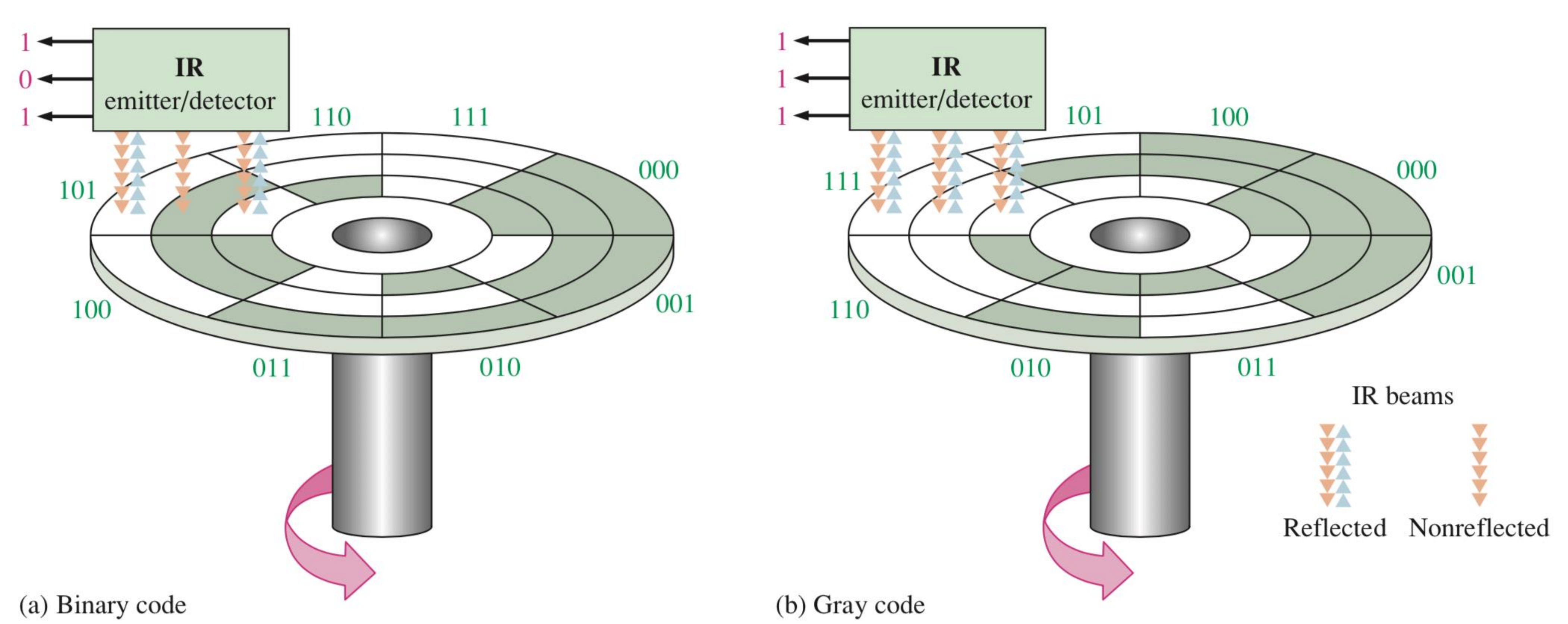
▪ 184:	0001^{1}	1000^{1}	0100
▪ 576:	<u>0101</u>	<u>0111</u>	<u>0110</u>
▪ Binary Sum:	0111	10000	1010
▪ Add 6		<u>0110</u>	<u>0110</u>
▪ BCD Sum :	0111	0110	0000

GRAY CODES

- Only one bit in the code group changes (or toggles) in going from one number to the next.
- Used in applications in which the normal sequence of binary numbers generated by the hardware may produce an error or ambiguity during the transition from one number to the next.
  - Example: Number changing from 0111 to 1000 may produce an intermediate erroneous number 1001 if the value of the rightmost bit takes longer to change.

Gray Code

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15



GRAY CODES

In binary, if one beam is slightly ahead of the others during the transition from one sector to the next, an erroneous output can occur. In gray, even though the beams may not be in precise alignment, there will never be a transitional error.

ASCII CODES

Many applications of digital computers require the handling not only of numbers, but also of other characters or symbols, such as the letters of the alphabet.

- Codes that represent 0 to 9 and A to Z and special characters like $, space, " etc.
- Code represents both uppercase and lowercase letters as well.

The standard binary codes for alphanumeric characters is the American Standard Code for Information Interchange (ASCII) → uses 7-bits to code 128 characters

American Standard Code for Information Interchange (ASCII).

Control Characters				**Graphic Symbols**											
Name	**Dec**	**Binary**	**Hex**	**Symbol**	**Dec**	**Binary**	**Hex**	**Symbol**	**Dec**	**Binary**	**Hex**	**Symbol**	**Dec**	**Binary**	**Hex**
NUL	0	0000000	00	space	32	0100000	20	@	64	1000000	40	′	96	1100000	60
SOH	1	0000001	01	!	33	0100001	21	A	65	1000001	41	a	97	1100001	61
STX	2	0000010	02	”	34	0100010	22	B	66	1000010	42	b	98	1100010	62
ETX	3	0000011	03	#	35	0100011	23	C	67	1000011	43	c	99	1100011	63
EOT	4	0000100	04	$	36	0100100	24	D	68	1000100	44	d	100	1100100	64
ENQ	5	0000101	05	%	37	0100101	25	E	69	1000101	45	e	101	1100101	65
ACK	6	0000110	06	&	38	0100110	26	F	70	1000110	46	f	102	1100110	66
BEL	7	0000111	07	’	39	0100111	27	G	71	1000111	47	g	103	1100111	67
BS	8	0001000	08	(	40	0101000	28	H	72	1001000	48	h	104	1101000	68
HT	9	0001001	09	)	41	0101001	29	I	73	1001001	49	i	105	1101001	69
LF	10	0001010	0A	*	42	0101010	2A	J	74	1001010	4A	j	106	1101010	6A
VT	11	0001011	0B	+	43	0101011	2B	K	75	1001011	4B	k	107	1101011	6B
FF	12	0001100	0C	,	44	0101100	2C	L	76	1001100	4C	l	108	1101100	6C
CR	13	0001101	0D	−	45	0101101	2D	M	77	1001101	4D	m	109	1101101	6D
SO	14	0001110	0E	.	46	0101110	2E	N	78	1001110	4E	n	110	1101110	6E
SI	15	0001111	0F	/	47	0101111	2F	O	79	1001111	4F	o	111	1101111	6F
DLE	16	0010000	10	0	48	0110000	30	P	80	1010000	50	p	112	1110000	70
DC1	17	0010001	11	1	49	0110001	31	Q	81	1010001	51	q	113	1110001	71
DC2	18	0010010	12	2	50	0110010	32	R	82	1010010	52	r	114	1110010	72
DC3	19	0010011	13	3	51	0110011	33	S	83	1010011	53	s	115	1110011	73
DC4	20	0010100	14	4	52	0110100	34	T	84	1010100	54	t	116	1110100	74
NAK	21	0010101	15	5	53	0110101	35	U	85	1010101	55	u	117	1110101	75
SYN	22	0010110	16	6	54	0110110	36	V	86	1010110	56	v	118	1110110	76
ETB	23	0010111	17	7	55	0110111	37	W	87	1010111	57	w	119	1110111	77
CAN	24	0011000	18	8	56	0111000	38	X	88	1011000	58	x	120	1111000	78
EM	25	0011001	19	9	57	0111001	39	Y	89	1011001	59	y	121	1111001	79
SUB	26	0011010	1A	:	58	0111010	3A	Z	90	1011010	5A	z	122	1111010	7A
ESC	27	0011011	1B	;	59	0111011	3B	[	91	1011011	5B	{	123	1111011	7B
FS	28	0011100	1C	<	60	0111100	3C	\	92	1011100	5C	|	124	1111100	7C
GS	29	0011101	1D	=	61	0111101	3D	]	93	1011101	5D	}	125	1111101	7D
RS	30	0011110	1E	>	62	0111110	3E	^	94	1011110	5E	~	126	1111110	7E
US	31	0011111	1F	?	63	0111111	3F	_	95	1011111	5F	Del	127	1111111	7F

ERROR DETECTION CODES

- To detect errors in data communication and processing, an eighth bit is sometimes added to the ASCII character to indicate its parity.

> A *parity bit* is an extra bit on a message to make the total number of 1's either even or odd.

- Parity helps in detecting errors during the transmission of information.
- The transmitter and receiver agree on the same protocols (even or odd number of 1's).

	With even parity	With odd parity
ASCII A = 1000001	01000001	11000001
ASCII T = 1010100	11010100	01010100

END OF PART 1 REVIEW

- Assuming all of the following are 2's complement signed numbers, what is the value of:

1. 00110010 in decimal, octal and hexadecimal $(50)_{10}$, $(062)_{8}$, $(32)_{16}$
2. 1110 in decimal $(-2)_{10}$
3. $(-41)_{10}$ in 8-bit, and 10-bit binary
   00101001 → 41
   11010111 → -41 in 8 bits
   1111010111 → -41 in 10 bits
4. $(8)_{10}$ and $(-8)_{10}$ in 4-bits binary 1000, 1000
5. $(-9)_{10}$ in 4-bits binary Cannot be expressed in 4 bits